

CODEGUARD AI

Intelligent AI-Powered Code Analysis, Security Detection and Automated
Remediation Platform

PROJECT REPORT

Technology Stack:

FastAPI • Streamlit • LangGraph • LangChain • Ollama
ChromaDB • PostgreSQL • SQLAlchemy • Alembic • Docker

August 2026

Abstract

CodeGuard AI is an AI-assisted software engineering platform designed to analyze source code, identify security and quality issues, explain findings in developer-friendly language, and propose safer remediation. The platform combines deterministic software-engineering components with retrieval-augmented generation (RAG), local large language models, vector search, and a multi-agent workflow. The objective is not to replace established static analysis or security tooling, but to add contextual reasoning, prioritization, explanation and remediation support around the findings that developers need to understand and act on.

The current implementation uses FastAPI as the backend service layer and Streamlit as the interactive user interface. PostgreSQL stores project and file metadata, while ChromaDB provides vector retrieval for the knowledge base. Ollama hosts local language and embedding models, reducing dependence on externally billed inference APIs. LangChain provides the LLM/RAG integration layer, and LangGraph orchestrates the specialized agents. SQLAlchemy 2.0 provides ORM access and Alembic supports database migrations. Report generation is implemented with Markdown/HTML/PDF-oriented components, with Docker and Docker Compose providing a reproducible deployment model.

The knowledge base contains 15 reference PDFs covering sources such as OWASP, CWE, CERT guidance, Python and Java security material. The indexing validation processed 258 pages into 737 chunks. The analysis workflow includes upload/scanning, code analysis, security analysis, retrieval, AI remediation, summarization and conversational assistance. Validation scenarios demonstrated detection of representative issues including SQL injection, unsafe eval usage, command injection and hardcoded secrets. These results establish a functional proof of concept and a strong base for further evaluation using benchmark datasets, precision/recall measurements, false-positive analysis, patch validation and production hardening.

Keywords: AI code review, secure coding, RAG, LangGraph, LangChain, Ollama, ChromaDB, OWASP, CWE, automated remediation, FastAPI, Streamlit, DevSecOps.

1. Introduction

Modern development teams operate under pressure to deliver software quickly while maintaining security, reliability and maintainability. Traditional code review is valuable but expensive in engineering time, and conventional static analysis can produce large volumes of findings that require human interpretation. CodeGuard AI addresses this gap by creating a single workflow in which code can be submitted, scanned, analyzed, enriched with trusted security knowledge, summarized and accompanied by remediation guidance.

The central design principle is a hybrid architecture. CodeGuard AI does not depend on an LLM alone to decide whether code is safe. Deterministic scanning and structured analysis provide evidence, while retrieval provides domain context, and the LLM provides explanation and reasoning. This separation is important because security decisions should be traceable to source-code evidence and recognized security guidance rather than generated from free-form model intuition.

A second design principle is modularity. The system is organized around specialized responsibilities rather than a single large prompt. Agents can be independently tested and improved. This makes it easier to add new scanners, knowledge sources, languages, remediation strategies and evaluation datasets without rewriting the complete application.

A third principle is local-first AI. Ollama enables local model execution, and the project uses a local embedding path rather than making the complete workflow dependent on paid external APIs. This is useful for source-code privacy, controlled experimentation and predictable development costs. The trade-off is that local inference performance depends on available CPU/GPU resources and model size.

The project is currently best characterized as a functional AI engineering prototype with a production-oriented architecture. It demonstrates end-to-end flow, database persistence, RAG indexing, multi-agent orchestration

and security/remediation use cases. The next level of maturity requires systematic benchmark evaluation, stronger patch verification, authentication/authorization, observability, queue-based execution, model evaluation and secure sandboxing for any code execution.

2. Problem Statement

Software security problems are often discovered late because code review, security review and testing are separated from the developer's normal workflow. A developer may receive a scanner finding such as "SQL injection" or "command injection" without enough contextual explanation to understand the exploit path, why the code is vulnerable, how severe the issue is, and what change should be made without introducing another defect.

Another challenge is information overload. Security standards such as OWASP and CWE contain extensive guidance, but manually searching these sources for every finding slows down review. An AI system can reduce this friction by retrieving relevant guidance at analysis time and grounding the generated explanation in those documents.

The problem is therefore defined as: **How can an AI-assisted platform analyze source code, detect security and quality risks, retrieve relevant secure-coding knowledge, prioritize findings, and produce actionable remediation while preserving traceability and developer control?**

The solution must also consider practical engineering constraints: source-code privacy, changing project requirements, incomplete datasets, model hallucination, false positives, explainability, extensibility and reproducible deployment.

3. Objectives and Scope

3.1 Primary objectives

- Accept source-code projects through a controlled upload workflow and maintain project/file metadata.
- Scan uploaded source code and identify analyzable files before deeper processing.
- Perform general code-quality and security analysis with specialized AI agents.
- Ground security explanations using a retrieval-augmented knowledge base.
- Use a local LLM and local embedding model through Ollama where practical.
- Generate remediation suggestions that are understandable and actionable for developers.
- Produce an executive-style summary and report output for review.
- Provide an interactive conversational interface for follow-up questions about findings.
- Maintain modular services so new languages, scanners and agents can be integrated later.
- Provide a Docker-based deployment path for repeatable setup.

3.2 Scope

The current scope focuses on source-code analysis rather than full software supply-chain security. It covers code ingestion, scanning, security reasoning, RAG retrieval, remediation suggestions, summaries and project-level reporting. The system is designed as an assistant to human developers and security reviewers. It is not intended to autonomously approve production releases or execute untrusted source code without additional isolation controls.

3.3 Out of scope for the current prototype

- Guaranteed zero false positives or zero false negatives.

- Autonomous deployment of generated patches to production.
- Execution of untrusted code on the host machine.
- Complete SAST coverage for every programming language.
- Formal verification of arbitrary software.
- Replacement of organizational security governance or human approval.

4. Functional and Non-Functional Requirements

4.1 Functional requirements

Requirement	Description	Status
Project ingestion	Create a project workspace and register uploaded files.	Implemented
Source scanning	Identify supported source files and prepare them for analysis.	Implemented / evolving
Code analysis	Generate structured observations on code quality and correctness.	Implemented
Security analysis	Detect representative security vulnerabilities and classify findings.	Implemented
Knowledge retrieval	Retrieve relevant security guidance from indexed documents.	Implemented
Remediation	Generate safer replacement patterns and explanations.	Implemented / evolving
Summary	Produce concise project-level analysis summaries.	Implemented
Chat assistant	Allow contextual questions about project findings.	Implemented
Persistence	Store projects/files and workflow metadata in PostgreSQL.	Implemented
Reporting	Generate review-oriented report output.	Implemented

4.2 Non-functional requirements

- Security: isolate uploaded content, avoid arbitrary execution, protect secrets and database credentials.
- Privacy: prefer local model inference for sensitive source code during development.
- Maintainability: use clear service/repository boundaries and migration-based database changes.
- Scalability: allow long-running analysis to move to asynchronous workers as the workload grows.
- Traceability: preserve file/project identity and retain evidence behind findings.
- Reliability: handle upload, parsing, model and retrieval failures without silently producing misleading results.
- Extensibility: support additional languages, scanners, knowledge sources and model providers.
- Reproducibility: package infrastructure with Docker Compose and configuration through environment variables.

5. System Architecture

CodeGuard AI follows a layered architecture in which the user interface communicates with a FastAPI backend. The backend coordinates persistence, workspace management, scanning and the AI workflow. PostgreSQL stores structured metadata, ChromaDB stores vectorized knowledge chunks, and Ollama provides local language/embedding model access. LangGraph coordinates the specialized agent workflow while LangChain provides model and retrieval abstractions.

5.1 Architecture components

Layer	Technology / Component	Responsibility
UI	Streamlit	Upload projects, display findings, summaries, remediation and chat.
API	FastAPI	REST endpoints, request validation and orchestration entry points.
Application	Services / repositories	Workspace, upload, scanning, persistence and business logic.
AI orchestration	LangGraph	Stateful multi-agent workflow and controlled transitions.
LLM framework	LangChain	Prompting, model integration, retrieval and chain utilities.
LLM / embeddings	Ollama	Local inference for language and embedding models.
Vector store	ChromaDB	Semantic retrieval over security knowledge chunks.
Relational DB	PostgreSQL	Project, file and application metadata.
ORM / migrations	SQLAlchemy 2.0 / Alembic	Database access and schema evolution.
Reports	ReportLab + Markdown/HTML	Human-readable report generation.
Deployment	Docker / Docker Compose	Reproducible multi-service environment.

5.2 High-level processing flow

- User creates/selects a project and uploads source code.
- Upload service creates an isolated workspace and records file metadata.
- Scanner identifies relevant source files and excludes unsupported or unsafe content.
- Code analysis and security agents inspect the source and produce structured observations.
- Security findings trigger retrieval from the indexed knowledge base.
- The remediation agent uses source evidence plus retrieved guidance to propose fixes.
- The summary agent consolidates results into a project-level report.
- The assistant/chat layer allows follow-up questions using the project context and knowledge base.

5.3 Workspace isolation

A project workspace is created using a unique project identifier. The design separates temporary processing data from report output and persistent application metadata. This reduces filename collisions and makes it possible to clean up intermediate artifacts independently. In a production deployment, workspace access should also be bound to an authenticated user or tenant and file paths should be normalized to prevent traversal attacks.

6. Data and Persistence Design

PostgreSQL is used for structured application state. The core entities include a project representation and project-file records. The Project entity provides a stable identity for an analysis workspace, while ProjectFile captures uploaded source metadata and its association with the parent project.

Entity	Purpose	Important concerns
Project	Represents an analysis workspace.	Unique identifier, timestamps, status and ownership in future versions.
ProjectFile	Represents a source file associated with a project.	Path safety, file type, size, checksum and processing state.

Finding	Recommended future persistent entity for security/code findings.	Severity, rule/CWE, evidence, confidence, location and remediation.
AnalysisRun	Recommended future entity for execution history.	Model/version, scanner version, duration, status and errors.

Alembic is used for schema migrations rather than relying on ad-hoc database changes. SQLAlchemy 2.0 provides a clear ORM boundary and makes repository-level testing easier. The database layer should remain independent from the LLM layer so that AI provider changes do not require schema redesign.

7. Upload and Scanning Pipeline

The upload subsystem is the first trust boundary. Source code is treated as untrusted input. The workflow therefore separates receiving files, validating them, creating a workspace, extracting or cloning content where applicable, scanning files, and handing only approved source artifacts to the analysis layer.

The current roadmap includes support for individual Python files, ZIP archives and GitHub repositories. ZIP processing requires archive validation and extraction controls. Repository ingestion requires controlled cloning and branch selection. In a hardened deployment, repository access should use network restrictions and explicit allowlists where organizational policy requires them.

- Validate filename, extension and content size before processing.
- Reject path traversal patterns and normalize all extracted paths.
- Apply project-level storage quotas.
- Avoid executing uploaded code during scanning.
- Record file metadata and processing status.
- Use checksums to detect duplicate content and improve auditability.
- Move long-running repository and archive processing to asynchronous workers as scale increases.

8. Security Analysis Engine

The security analysis layer combines source evidence with security knowledge. Representative validation scenarios demonstrated detection of SQL injection, unsafe eval usage, command injection and hardcoded secrets. These findings correspond to common vulnerability classes where the source itself provides strong evidence of unsafe behavior.

The platform should conceptually separate four attributes that are often incorrectly collapsed into a single severity value: **impact**, **exploitability**, **confidence**, and **business priority**. A high-impact issue with weak evidence should not be presented identically to a high-impact issue with strong source-level evidence. This separation is particularly important when AI-generated classifications are used.

8.1 Representative vulnerability classes

Finding type	Typical evidence	Recommended classification logic
SQL injection	User-controlled input reaches dynamically constructed SQL.	High/Critical depending on data access and exploitability; confidence increases with direct source evidence.
Unsafe eval	eval/exec receives dynamic or untrusted content.	High/Critical when attacker influence is plausible; document execution context.
Command injection	User-controlled content reaches shell execution.	High/Critical when shell metacharacters or uncontrolled arguments can be supplied.
Hardcoded secret	Credential/token/API key appears in source.	Severity depends on secret type, privilege, exposure and whether the value is live.

8.2 Severity model

A practical severity pipeline can combine deterministic rules with model-assisted contextual scoring. A simplified conceptual score is: **Risk = Impact × Exploitability × Confidence**, followed by business-context adjustment. This is a design principle rather than a claim that the current prototype uses a fixed mathematical scoring formula. For a review-ready production implementation, the exact rubric should be versioned and auditable.

Suggested severity bands are Critical, High, Medium, Low and Informational. The final label should always be accompanied by evidence, rationale and confidence. This makes the result explainable and prevents a generated label from appearing as an unexplained black-box decision.

9. Retrieval-Augmented Generation (RAG)

RAG is a core part of CodeGuard AI because general-purpose LLM knowledge is not sufficient for a security review. The system indexes trusted reference documents and retrieves relevant passages for a specific finding. This reduces the need for the model to reconstruct security guidance from memory and provides a path toward traceable explanations.

The knowledge base currently contains 15 PDFs from categories including OWASP, CWE, CERT and language-specific security guidance. The indexing validation processed 258 pages and generated 737 chunks. Chunking converts long documents into retrieval-sized units while preserving enough context for the model to reason about a vulnerability class.

The embedding pipeline uses a local embedding model through Ollama, with ChromaDB storing vectors and associated document metadata. The architecture is intentionally provider-independent so that the embedding model can be changed later without redesigning the application.

A typical RAG cycle is: query construction → semantic retrieval → relevance filtering → context assembly → grounded prompt → LLM generation → output validation. Production hardening should add source attribution, retrieval scores, context limits and explicit instructions not to invent unsupported security guidance.

9.1 RAG processing stages

Stage	Input	Output
Ingestion	PDF/reference document	Extracted text and metadata
Chunking	Document text	Retrieval-sized chunks
Embedding	Chunk text	Vector representation
Indexing	Vectors + metadata	ChromaDB collection
Query	Finding/source context	Semantic query vector
Retrieval	Query vector	Top relevant chunks
Generation	Finding + retrieved context	Grounded explanation/remediation

10. Multi-Agent Architecture

The project uses LangGraph to orchestrate specialized agents. The purpose of multiple agents is separation of concerns: upload handling should not have to understand security remediation, and a summary agent should not have to perform raw source scanning. A graph-based design also provides a clear place to add validation gates and fallback behavior.

10.1 Agent responsibilities

Agent	Primary responsibility	Expected artifact
Upload Agent	Coordinate project/file ingestion.	Validated project/file state.
Code Analysis Agent	Review code structure, quality and potential defects.	Structured code observations.
Security Agent	Identify security vulnerabilities and contextual risk.	Security findings with evidence.
Embedding Agent	Prepare and index knowledge content.	Indexed knowledge chunks.
AI Fix / Remediation Agent	Suggest safer code and explain the change.	Remediation proposal.
Summary Agent	Consolidate findings into an executive summary.	Project-level summary.
Chat Agent	Answer contextual user questions.	Conversational explanation.

The multi-agent design does not require a separate model for every agent. Agents can share the same local model while using different system instructions, tools, structured state and validation rules. This is more resource-efficient during development.

11. AI Remediation and Fallback Strategy

Remediation is the most sensitive part of an AI code-review system because a plausible-looking fix can still be wrong. CodeGuard AI therefore treats generated remediation as a proposal rather than an automatically trusted patch. The remediation agent should receive the vulnerable code context, finding details, retrieved secure-coding guidance and explicit constraints.

A robust remediation cycle is: identify finding → retrieve guidance → generate candidate fix → perform static consistency checks → compare before/after behavior where safe → present the patch for human approval. If any middle step fails, the system should fall back to an explanation-only response rather than fabricating a confident fix.

Failure condition	Safe fallback
LLM unavailable	Return deterministic scanner findings and retry later.
Retriever unavailable	Explain finding with local rule metadata; mark knowledge grounding unavailable.
Low retrieval relevance	Do not present weak context as authoritative; request human review.
Generated patch fails validation	Show finding and explain why no patch was accepted.
Parsing failure	Report the affected file and continue with other supported files.

12. Analysis Visualization and Radar-Style Scoring

The project includes analysis visualization intended to provide a compact view of multiple quality/security dimensions. A radar chart can represent normalized scores such as security, maintainability, reliability, performance and best-practice adherence. Each axis must use the same direction and scale, typically 0–100, so that a larger area does not accidentally imply a mathematically stronger result.

The chart should be treated as a communication aid rather than a scientific measurement by itself. Each plotted score should have an explicit definition and aggregation rule. For example, a security score could be derived from weighted finding severity and confidence; maintainability could incorporate code smells and complexity observations; and best-practice adherence could use rule-based checks. The exact formula should be versioned so that scores remain comparable between analysis runs.

13. Reporting Layer

The reporting layer converts machine-generated findings into a review-oriented artifact. A good report should answer five questions: what was analyzed, what was found, why it matters, how it can be fixed, and what should happen next. Report sections should distinguish evidence from generated interpretation.

The final report format can include an executive summary, project metadata, findings table, severity distribution, security analysis, code-quality observations, RAG references, remediation proposals, limitations and future scope. PDF generation through ReportLab provides a deterministic artifact suitable for manager review and archival.

14. Testing and Validation

Validation was performed at both component and workflow levels. The project includes test coverage for the RAG workflow and specialized AI agents, including code analysis, security analysis, remediation, summary and assistant behavior. The knowledge-base build was also validated using the reference PDF collection.

14.1 Validation results

Test area	Validation focus	Observed outcome
Knowledge base	Document loading, chunk creation and indexing.	258 pages processed; 737 chunks created from 15 PDFs in the validation run.
Security agent	Representative vulnerability patterns.	Detected SQL injection, unsafe eval, command injection and hardcoded secret scenarios.
RAG	Context retrieval for security reasoning.	Functional retrieval pipeline established; relevance evaluation should be expanded.
Remediation	Generate safer alternatives.	Functional proposal workflow; patch verification remains a key maturity area.
Summary agent	Aggregate findings into readable output.	Functional summary workflow.
Assistant	Answer contextual questions.	Functional conversational workflow; grounding and refusal tests should be expanded.

14.2 Recommended quantitative evaluation

- Precision, recall and F1 for vulnerability detection against a labeled benchmark.
- False-positive rate per rule/vulnerability class.
- Top-k retrieval recall and context relevance for the RAG pipeline.
- Remediation acceptance rate after expert review.
- Patch correctness rate using regression/security test suites.
- Latency and inference cost per file and per project.
- Failure rate under malformed uploads, model outages and retrieval failures.
- Consistency of severity classifications across repeated runs.

15. Security, Privacy and Threat Model

Because CodeGuard AI processes source code, the platform itself must be designed as a security-sensitive application. The most important threat is treating uploaded source code as trusted content. Files may contain secrets, malware-like payloads, archive bombs, path traversal attempts or intentionally adversarial prompts designed to manipulate the model.

Threat	Risk	Mitigation direction
Path traversal	Files escape workspace.	Normalize paths; reject traversal; enforce workspace root.
Archive bomb	Resource exhaustion.	Size/file-count limits; extraction quotas; reject nested archives.
Prompt injection in source	Model follows instructions embedded in code/comments.	Treat source as untrusted data; delimit content; enforce system policy.
Secret leakage	Credentials appear in reports/logs.	Redaction; structured logging; avoid persisting raw secrets unnecessarily.
Code execution	Malicious code runs on host.	Never execute uploads by default; sandbox any future dynamic execution.
Dependency compromise	Third-party package risk.	Pin versions; scan dependencies; use trusted registries and lockfiles.
Threat	Risk	Mitigation direction
Database exposure	Project metadata/source metadata leaks.	Authentication, least privilege, encrypted transport and backups.

16. Key Engineering Decisions and Trade-offs

Decision	Reason	Trade-off
FastAPI backend	Strong Python async/web ecosystem and clear API contracts.	Requires disciplined separation between sync model calls and API execution.
Streamlit UI	Rapid development of an interactive technical prototype.	Less flexible than a dedicated SPA for complex production UX.
PostgreSQL	Reliable relational persistence and mature tooling.	Requires operational database management.
ChromaDB	Simple local vector-store integration for RAG development.	Production scale and operational controls may require reassessment.
Ollama	Local LLM/embedding execution and privacy benefits.	Inference quality/speed depends on local hardware and model.
LangGraph	Explicit multi-step agent state and workflow.	Graph complexity increases as more branches and tools are added.
Docker Compose	Repeatable local/development deployment.	Production orchestration requires stronger secrets/network/resource management.

17. Current Limitations

- The prototype has not yet established production-grade precision/recall metrics across a large labeled dataset.
- LLM output remains probabilistic and must be validated before automated use.
- Generated remediation can be incomplete or contextually incorrect without test-based patch verification.
- Language coverage is limited compared with mature commercial SAST platforms.
- RAG quality depends on the quality, freshness and coverage of the indexed documents.
- Local model performance varies with available compute resources.
- Authentication, multi-tenancy, audit controls and enterprise access policies require further implementation for production deployment.
- Long-running analysis is better suited to background workers and job queues than a synchronous request lifecycle.
- Any future dynamic code execution must be isolated in a hardened sandbox.

18. Future Scope and Roadmap

Phase	Enhancement	Expected value
1	Benchmark evaluation and labeled datasets.	Objective measurement of detection and false-positive performance.
2	AST/CFG/data-flow integration.	More deterministic evidence and better vulnerability context.
3	Patch verification pipeline.	Reduce unsafe or incomplete AI remediation.
4	Background job queue.	Scalable analysis for larger repositories.
5	Authentication and RBAC.	Secure team/enterprise usage.
6	Observability.	Metrics, tracing, model latency and failure visibility.
7	More languages/scanners.	Broader engineering coverage.
8	CI/CD integration.	Shift security review closer to pull requests and builds.
9	Security knowledge refresh.	Keep OWASP/CWE/CERT and internal policies current.
10	Enterprise governance.	Audit trails, policy enforcement, approvals and tenant isolation.

19. Conclusion

CodeGuard AI demonstrates a practical approach to AI-assisted secure software engineering by combining deterministic application components with retrieval, local LLM inference and multi-agent orchestration. The architecture separates ingestion, scanning, analysis, retrieval, remediation, summarization and conversation, making the platform easier to test and extend than a monolithic LLM prompt.

The strongest aspect of the current implementation is the end-to-end engineering foundation: FastAPI provides a service boundary, PostgreSQL provides persistence, ChromaDB provides semantic retrieval, Ollama supports local AI execution, LangGraph manages workflow state, and Docker supports reproducibility. The knowledge-base validation and representative vulnerability tests demonstrate that the system is capable of meaningful analysis rather than being only a UI prototype.

The most important next step is rigorous evaluation. A security product should not be judged only by impressive examples; it must be measured using labeled datasets, false-positive analysis, retrieval relevance, patch correctness and repeatability. The project is therefore ready for final manager review as a strong prototype, while the roadmap clearly identifies the work required for production maturity.

Appendix A: Technology Stack

Category	Technology	Role
Backend	FastAPI	API layer and service endpoints.
Frontend	Streamlit	Interactive analysis dashboard and user workflow.
Language	Python	Core implementation language.
AI orchestration	LangGraph	Multi-agent stateful workflow.
AI framework	LangChain	LLM/RAG integration.
LLM	Ollama	Local language-model serving.
Embeddings	Ollama / local embedding model	Semantic representation of knowledge chunks.

Vector DB	ChromaDB	Vector indexing and retrieval.
Relational DB	PostgreSQL	Application metadata persistence.
ORM	SQLAlchemy 2.0	Database abstraction.
Migrations	Alembic	Schema versioning.
Reporting	ReportLab / Markdown / HTML	Report generation.
Deployment	Docker / Docker Compose	Environment packaging and orchestration.

Appendix B: Suggested Review Checklist

- Verify the current repository structure and confirm that the report matches the latest implementation.
- Confirm all implemented agents and API endpoints against the latest code.
- Run the complete automated test suite before submission.
- Validate the knowledge-base index count after the final document refresh.
- Check that secrets and environment variables are excluded from screenshots and reports.
- Review representative vulnerability findings manually for correctness.
- Review remediation suggestions manually and do not auto-apply generated fixes.
- Confirm Docker Compose startup from a clean environment.
- Record model versions and scanner versions for reproducibility.
- Add benchmark metrics before describing the system as production-ready.

Appendix C: Reference Knowledge Categories

The current knowledge-base design uses trusted security and language-development references, with emphasis on OWASP, CWE, CERT-oriented guidance, Python security and Java security material. For a production release, the exact document versions, publication dates, hashes and source URLs should be stored as metadata so that every generated recommendation can be traced to a specific knowledge snapshot.

Recommended governance practice is to treat the knowledge base as versioned data. When a source document changes, rebuild the affected embeddings, record the index version, and retain enough metadata to reproduce an earlier analysis. This is particularly important for security guidance, where recommendations and vulnerability information evolve over time.

Appendix D: Final Review Note

This report is a manager-review draft. Before formal academic, client or production submission, the implementation-specific details should be cross-checked against the final repository commit, final screenshots, final test results and final deployment configuration. Any metric that is not produced by an automated test or benchmark should be described as an engineering observation rather than a measured performance claim.